Module 9.3 - Template-Driven Forms: Advanced Features

> **Learning Objectives:**
>
> - Prepopulate form fields with existing data
> - Understand one-way data binding in forms
> - Group related form controls using ngModelGroup
> - Handle form submission properly
> - Reset and manage form states programmatically
> - Build complex forms with nested data structures

Table of Contents

1. Prepopulating Form Fields

1.1 Why Prepopulate Forms?

In many scenarios, forms need to display existing data that users can view or edit. Common use cases include:

- **Edit Mode:** Loading existing user information for updates
- **Default Values:** Setting sensible defaults for new entries
- **Draft Recovery:** Restoring previously entered data
- **Profile Management:** Displaying current user settings

1.2 Basic Prepopulation

Prepopulating forms in Angular is straightforward using property binding with `ngModel`. The data flows from the component class to the template.

Component (user-form.component.ts):

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-user-form',
  templateUrl: './user-form.component.html'
})
export class UserFormComponent {
  // Define properties with initial values
  userName: string = 'John Doe';
  userEmail: string = 'john@example.com';
  userAge: number = 25;

  onSubmit() {
    console.log('Form Submitted:', {
      name: this.userName,
      email: this.userEmail,
      age: this.userAge
    });
  }
}
```

Template (user-form.component.html):

```
<form #userForm="ngForm" (ngSubmit)="onSubmit()">
  <div>
    <label>Name:</label>
    <input type="text"
           name="userName"
           [(ngModel)]="userName"
           required>
  </div>

  <div>
    <label>Email:</label>
    <input type="email"
           name="userEmail"
           [(ngModel)]="userEmail"
           required>
  </div>

  <div>
    <label>Age:</label>
    <input type="number"
           name="userAge"
           [(ngModel)]="userAge"
           required>
  </div>

  <button type="submit">Submit</button>
</form>
```

Output:

When the component loads, the form displays: - Name field: "John Doe" - Email field: "john@example.com" - Age field: "25" These values are editable and changes update the component properties immediately.

1.3 Prepopulating with Object Data

In real applications, data typically comes from objects rather than individual properties. This approach is more organized and scalable.

Component:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-edit-profile',
  templateUrl: './edit-profile.component.html'
})
export class EditProfileComponent {
  // User object with existing data
  user = {
    firstName: 'Sarah',
    lastName: 'Johnson',
    email: 'sarah.johnson@example.com',
    phone: '555-0123'
  };

  onUpdate() {
    console.log('Updated User:', this.user);
  }
}
```

Template:

```
<form #profileForm="ngForm" (ngSubmit)="onUpdate()">
  <div>
    <label>First Name:</label>
    <input type="text"
           name="firstName"
           [(ngModel)]="user.firstName"
           required>
  </div>

  <div>
    <label>Last Name:</label>
    <input type="text"
           name="lastName"
           [(ngModel)]="user.lastName"
           required>
  </div>

  <div>
    <label>Email:</label>
    <input type="email"
```

```
            name="email"
            [(ngModel)]="user.email"
            required>
  </div>

  <div>
    <label>Phone:</label>
    <input type="tel"
           name="phone"
           [(ngModel)]="user.phone">
  </div>

  <button type="submit" [disabled]="!profileForm.valid">
    Update Profile
  </button>
</form>
```

Output:

Form displays pre-filled with: - First Name: "Sarah" - Last Name: "Johnson" - Email: "sarah.johnson@example.com" - Phone: "555-0123" User can modify any field and click "Update Profile" when done.

1.4 Loading Data from Services

Typically, form data comes from services. Here's how to load and prepopulate data:

Service (user.service.ts):

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class UserService {
  // Simulates getting user data
  getUserById(id: number) {
    return {
      id: id,
      name: 'Alice Cooper',
      email: 'alice@example.com',
      bio: 'Software Developer'
```

```
    };
  }
}
```

Component:

```
import { Component, OnInit } from '@angular/core';
import { UserService } from './user.service';

@Component({
  selector: 'app-edit-user',
  templateUrl: './edit-user.component.html'
})
export class EditUserComponent implements OnInit {
  user: any = {
    name: '',
    email: '',
    bio: ''
  };

  constructor(private userService: UserService) {}

  ngOnInit() {
    // Load user data when component initializes
    this.user = this.userService.getUserById(1);
  }

  onSave() {
    console.log('Saving user:', this.user);
  }
}
```

Template:

```
<form #editForm="ngForm" (ngSubmit)="onSave()">
  <div>
    <label>Name:</label>
    <input type="text"
           name="name"
           [(ngModel)]="user.name"
           required>
  </div>
```

```
  <div>
    <label>Email:</label>
    <input type="email"
           name="email"
           [(ngModel)]="user.email"
           required>
  </div>

  <div>
    <label>Bio:</label>
    <textarea name="bio"
              [(ngModel)]="user.bio"
              rows="4"></textarea>
  </div>

  <button type="submit" [disabled]="!editForm.valid">
    Save Changes
  </button>
</form>
```

Output:

When component loads (ngOnInit runs): 1. Service fetches user data for ID 1 2. Form displays: - Name: "Alice Cooper" - Email: "alice@example.com" - Bio: "Software Developer" 3. User can edit and save changes

---

2. One-Way Data Binding in Forms

2.1 Understanding One-Way Binding

While two-way binding (`[(ngModel)]`) is convenient, sometimes you need one-way binding to:

- **Display-only fields:** Show data without allowing direct modification
- **Controlled updates:** Update model only on specific events
- **Validation before update:** Process input before updating the model
- **Performance optimization:** Reduce change detection cycles

2.2 Property Binding (Model to View)

Use `[ngModel]` to bind data from component to template without automatic updates back:

Component:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-display-form',
  templateUrl: './display-form.component.html'
})
export class DisplayFormComponent {
  productName: string = 'Laptop';
  productPrice: number = 999.99;

  // Manual update method
  updatePrice(newPrice: string) {
    this.productPrice = parseFloat(newPrice);
    console.log('Price updated to:', this.productPrice);
  }
}
```

Template:

```
<div>
  <h3>Product: {{ productName }}</h3>

  <!-- One-way binding: displays value but doesn't update productPrice -->
  <label>Price (Display Only):</label>
  <input type="number"
         [ngModel]="productPrice"
         name="displayPrice"
         disabled>

  <!-- Manual update with template reference variable -->
  <label>New Price:</label>
  <input type="number"
         #priceInput
         [value]="productPrice">
  <button (click)="updatePrice(priceInput.value)">
    Update Price
  </button>
```

```
  <p>Current Price: ${{ productPrice }}</p>
</div>
```

Output:

Initial display: - Product: Laptop - Price (Display Only): 999.99 (disabled field) - New Price: 999.99 - Current Price: $999.99 User changes New Price to 1299.99 and clicks "Update Price": - Console: "Price updated to: 1299.99" - Display updates to show $1299.99

2.3 Event Binding (View to Model)

Use `(ngModelChange)` to capture changes without two-way binding:

Component:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-search-form',
  templateUrl: './search-form.component.html'
})
export class SearchFormComponent {
  searchTerm: string = '';
  searchResults: string[] = [];

  onSearchChange(value: string) {
    this.searchTerm = value;
    console.log('Searching for:', value);

    // Simulate search logic
    if (value.length >= 3) {
      this.searchResults = [
        `Result 1 for "${value}"`,
        `Result 2 for "${value}"`,
        `Result 3 for "${value}"`
      ];
    } else {
      this.searchResults = [];
```

```
    }
  }
}
```

Template:

```
<div>
  <label>Search:</label>
  <input type="text"
         [ngModel]="searchTerm"
         (ngModelChange)="onSearchChange($event)"
         name="search"
         placeholder="Type at least 3 characters">

  <div *ngIf="searchResults.length > 0">
    <h4>Search Results:</h4>
    <ul>
      <li *ngFor="let result of searchResults">{{ result }}</li>
    </ul>
  </div>

  <p *ngIf="searchTerm.length > 0 && searchTerm.length < 3">
    Please enter at least 3 characters to search
  </p>
</div>
```

Output:

User types "ang": - Console: "Searching for: ang" - Displays: * Result 1 for "ang" * Result 2 for "ang" * Result 3 for "ang" User types "an" (deletes one character): - Console: "Searching for: an" - Message: "Please enter at least 3 characters to search"

2.4 Combining Property and Event Binding

You can use both bindings separately instead of two-way binding:

```
<!-- Two-way binding (shorthand) -->
<input [(ngModel)]="username" name="username">

<!-- Equivalent one-way bindings -->
```

```
<input [ngModel]="username"
       (ngModelChange)="username = $event"
       name="username">
```

This gives you control to add logic during the update:

```
<input [ngModel]="username"
       (ngModelChange)="onUsernameChange($event)"
       name="username">
```

Component method:

```
onUsernameChange(value: string) {
  // Add custom logic before updating
  this.username = value.toLowerCase(); // Force lowercase
  console.log('Username changed to:', this.username);
}
```

---

3. Grouping Controls with ngModelGroup

3.1 Why Group Form Controls?

As forms grow complex, organizing related fields becomes essential. `ngModelGroup` provides:

- **Logical Organization:** Group related fields (address, contact info)
- **Nested Data Structures:** Create hierarchical form data
- **Grouped Validation:** Validate sections independently
- **Cleaner Code:** Better form structure and maintainability

3.2 Basic ngModelGroup Usage

Component:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-registration',
  templateUrl: './registration.component.html'
})
export class RegistrationComponent {
  onSubmit(form: any) {
    console.log('Form Value:', form.value);
    console.log('Personal Info:', form.value.personal);
    console.log('Contact Info:', form.value.contact);
  }
}
```

Template:

```
<form #regForm="ngForm" (ngSubmit)="onSubmit(regForm)">
  <!-- Personal Information Group -->
  <fieldset ngModelGroup="personal">
    <legend>Personal Information</legend>

    <div>
      <label>First Name:</label>
      <input type="text"
             name="firstName"
             ngModel
             required>
    </div>

    <div>
      <label>Last Name:</label>
      <input type="text"
             name="lastName"
             ngModel
             required>
    </div>

    <div>
      <label>Date of Birth:</label>
      <input type="date"
             name="dob"
             ngModel>
    </div>
  </fieldset>
```

```
  <!-- Contact Information Group -->
  <fieldset ngModelGroup="contact">
    <legend>Contact Information</legend>

    <div>
      <label>Email:</label>
      <input type="email"
             name="email"
             ngModel
             required>
    </div>

    <div>
      <label>Phone:</label>
      <input type="tel"
             name="phone"
             ngModel>
    </div>
  </fieldset>

  <button type="submit" [disabled]="!regForm.valid">
    Register
  </button>
</form>
```

Output:

When user fills and submits: - First Name: "Michael" - Last Name: "Brown" - Date of Birth: "1990-05-15" - Email: "michael@example.com" - Phone: "555-9876" Console output: Form Value: { personal: { firstName: "Michael", lastName: "Brown", dob: "1990-05-15" }, contact: { email: "michael@example.com", phone: "555-9876" } }

3.3 Accessing Group Validation State

You can check validation status of specific groups:

Template:

```
<form #orderForm="ngForm">
  <fieldset ngModelGroup="shipping" #shippingGroup="ngModelGroup">
    <legend>Shipping Address</legend>
```

```
    <div>
      <label>Street:</label>
      <input type="text"
             name="street"
             ngModel
             required
             minlength="5">
    </div>

    <div>
      <label>City:</label>
      <input type="text"
             name="city"
             ngModel
             required>
    </div>

    <div>
      <label>Zip Code:</label>
      <input type="text"
             name="zip"
             ngModel
             required
             pattern="[0-9]{5}">
    </div>

    <!-- Display group validation status -->
    <div *ngIf="shippingGroup.invalid && shippingGroup.touched"
         style="color: red;">
      Please complete all shipping address fields correctly
    </div>
  </fieldset>

  <fieldset ngModelGroup="billing" #billingGroup="ngModelGroup">
    <legend>Billing Address</legend>

    <div>
      <label>Card Number:</label>
      <input type="text"
             name="cardNumber"
             ngModel
             required
             pattern="[0-9]{16}">
    </div>

    <div>
```

```
      <label>CVV:</label>
      <input type="text"
             name="cvv"
             ngModel
             required
             pattern="[0-9]{3}">
    </div>

    <div *ngIf="billingGroup.invalid && billingGroup.touched"
         style="color: red;">
      Please complete all billing information correctly
    </div>
  </fieldset>

  <button type="submit" [disabled]="!orderForm.valid">
    Place Order
  </button>
</form>
```

Output:

Validation behavior: 1. User touches shipping street field and leaves it empty → "Please complete all shipping address fields correctly" appears 2. User fills all shipping fields correctly → Shipping error message disappears 3. User enters invalid card number "123" (needs 16 digits) → "Please complete all billing information correctly" appears 4. Submit button remains disabled until both groups are valid

3.4 Nested Groups

Groups can be nested for complex forms:

```
<form #employeeForm="ngForm">
  <fieldset ngModelGroup="employee">
    <legend>Employee Details</legend>

    <input type="text" name="empId" ngModel required>

    <!-- Nested group for address -->
    <fieldset ngModelGroup="address">
      <legend>Address</legend>

      <input type="text" name="street" ngModel required>
```

```
      <input type="text" name="city" ngModel required>

      <!-- Further nested group for coordinates -->
      <fieldset ngModelGroup="location">
        <legend>GPS Coordinates</legend>
        <input type="number" name="latitude" ngModel>
        <input type="number" name="longitude" ngModel>
      </fieldset>
    </fieldset>
  </fieldset>
</form>
```

Output:

```
Resulting form structure: { employee: { empId: "E123", address: { street: "123 Main St", city: "Springfield", location: { latitude: 40.7128, longitude: -74.0060 } } } }
```

---

4. Form Submission Handling

4.1 Submit Event Handling

Angular provides the `(ngSubmit)` event for proper form submission handling. This event:

- Fires when form is submitted (button click or Enter key)
- Prevents default browser form submission
- Provides access to form data and validation state

Basic Submission:

Component:

```
import { Component } from '@angular/core';
import { NgForm } from '@angular/forms';

@Component({
  selector: 'app-contact',
  templateUrl: './contact.component.html'
})
```

```
export class ContactComponent {
  formSubmitted = false;

  onSubmit(contactForm: NgForm) {
    if (contactForm.valid) {
      console.log('Form Data:', contactForm.value);
      this.formSubmitted = true;

      // Process the form (e.g., send to service)
      console.log('Processing submission...');
    } else {
      console.log('Form is invalid');
    }
  }
}
```

Template:

```
<form #contactForm="ngForm" (ngSubmit)="onSubmit(contactForm)">
  <div>
    <label>Name:</label>
    <input type="text"
           name="name"
           ngModel
           required
           minlength="3">
  </div>

  <div>
    <label>Email:</label>
    <input type="email"
           name="email"
           ngModel
           required>
  </div>

  <div>
    <label>Message:</label>
    <textarea name="message"
              ngModel
              required
              minlength="10"></textarea>
  </div>

  <button type="submit" [disabled]="!contactForm.valid">
```

```
    Send Message
  </button>

  <div *ngIf="formSubmitted" style="color: green;">
    Thank you! Your message has been sent.
  </div>
</form>
```

Output:

User fills form: - Name: "Jennifer" - Email: "jennifer@example.com" - Message: "Hello, I need help with my order" Clicks "Send Message": - Console: Form Data: { name: "Jennifer", email: "jennifer@example.com", message: "Hello, I need help with my order" } - Console: Processing submission... - Green message appears: "Thank you! Your message has been sent."

4.2 Resetting Forms

After submission, you often need to reset the form:

Component:

```
import { Component } from '@angular/core';
import { NgForm } from '@angular/forms';

@Component({
  selector: 'app-feedback',
  templateUrl: './feedback.component.html'
})
export class FeedbackComponent {
  successMessage = '';

  onSubmit(form: NgForm) {
    if (form.valid) {
      console.log('Submitting:', form.value);

      // Simulate processing
      this.successMessage = 'Feedback submitted successfully!';

      // Reset form after submission
      form.reset();
```

```
      // Clear success message after 3 seconds
      setTimeout(() => {
        this.successMessage = '';
      }, 3000);
    }
  }
}
```

Template:

```
<form #feedbackForm="ngForm" (ngSubmit)="onSubmit(feedbackForm)">
  <div>
    <label>Rating:</label>
    <select name="rating" ngModel required>
      <option value="">Select rating</option>
      <option value="5">5 - Excellent</option>
      <option value="4">4 - Good</option>
      <option value="3">3 - Average</option>
      <option value="2">2 - Poor</option>
      <option value="1">1 - Very Poor</option>
    </select>
  </div>

  <div>
    <label>Comments:</label>
    <textarea name="comments" ngModel required></textarea>
  </div>

  <button type="submit" [disabled]="!feedbackForm.valid">
    Submit Feedback
  </button>

  <div *ngIf="successMessage" style="color: green; margin-top: 10px;">
    {{ successMessage }}
  </div>
</form>
```

Output:

```
User submits feedback: 1. Selects rating: "5 - Excellent" 2. Enters comments: "Great service!" 3. Clicks "Submit Feedback" Result: - Console: Submitting: { rating: "5", comments: "Great service!" } - Success message appears: "Feedback submitted successfully!" - Form fields clear (reset to empty) - After 3 seconds, success message disappears
```

4.3 Reset with Default Values

You can reset form with specific values:

Component:

```
import { Component } from '@angular/core';
import { NgForm } from '@angular/forms';

@Component({
  selector: 'app-settings',
  templateUrl: './settings.component.html'
})
export class SettingsComponent {
  defaultSettings = {
    theme: 'light',
    notifications: true,
    language: 'en'
  };

  onSave(form: NgForm) {
    console.log('Saving settings:', form.value);
  }

  onReset(form: NgForm) {
    // Reset to default values
    form.reset(this.defaultSettings);
    console.log('Reset to defaults');
  }
}
```

Template:

```
<form #settingsForm="ngForm">
  <div>
    <label>Theme:</label>
    <select name="theme" [(ngModel)]="defaultSettings.theme">
      <option value="light">Light</option>
      <option value="dark">Dark</option>
    </select>
  </div>

  <div>
    <label>
      <input type="checkbox"
             name="notifications"
             [(ngModel)]="defaultSettings.notifications">
      Enable Notifications
    </label>
  </div>

  <div>
    <label>Language:</label>
    <select name="language" [(ngModel)]="defaultSettings.language">
      <option value="en">English</option>
      <option value="es">Spanish</option>
      <option value="fr">French</option>
    </select>
  </div>

  <button type="button" (click)="onSave(settingsForm)">
    Save Settings
  </button>
  <button type="button" (click)="onReset(settingsForm)">
    Reset to Defaults
  </button>
</form>
```

Output:

```
User changes: - Theme: dark - Notifications: unchecked - Language: Spanish Clicks
"Reset to Defaults": - Form reverts to: * Theme: light * Notifications: checked *
Language: English - Console: "Reset to defaults"
```

4.4 Handling Submission Status

Track submission state for better UX:

Component:

```
import { Component } from '@angular/core';
import { NgForm } from '@angular/forms';

@Component({
  selector: 'app-newsletter',
  templateUrl: './newsletter.component.html'
})
export class NewsletterComponent {
  isSubmitting = false;
  submitSuccess = false;
  submitError = false;

  onSubscribe(form: NgForm) {
    if (form.valid) {
      this.isSubmitting = true;
      this.submitSuccess = false;
      this.submitError = false;

      console.log('Subscribing:', form.value.email);

      // Simulate API call delay
      setTimeout(() => {
        this.isSubmitting = false;

        // Simulate success (90% of the time)
        if (Math.random() > 0.1) {
          this.submitSuccess = true;
          form.reset();
          console.log('Subscription successful!');
        } else {
          this.submitError = true;
          console.log('Subscription failed');
        }
      }, 2000);
    }
  }
}
```

Template:

```
<form #subscribeForm="ngForm" (ngSubmit)="onSubscribe(subscribeForm)">
  <div>
    <label>Email Address:</label>
    <input type="email"
           name="email"
           ngModel
           required
           email>
  </div>

  <button type="submit"
          [disabled]="!subscribeForm.valid || isSubmitting">
    {{ isSubmitting ? 'Subscribing...' : 'Subscribe' }}
  </button>

  <div *ngIf="isSubmitting" style="color: blue;">
    Please wait...
  </div>

  <div *ngIf="submitSuccess" style="color: green;">
    Successfully subscribed! Check your email.
  </div>

  <div *ngIf="submitError" style="color: red;">
    Subscription failed. Please try again.
  </div>
</form>
```

Output:

User enters email: "user@example.com" and clicks Subscribe: During submission (2 seconds): - Button text: "Subscribing..." - Button disabled - Blue message: "Please wait..." After success: - Console: "Subscription successful!" - Green message: "Successfully subscribed! Check your email." - Form clears - Button re-enabled with text "Subscribe"

5. Practical Example: User Profile Editor

Let's build a complete profile editor that combines all concepts:

Service (profile.service.ts):

```
import { Injectable } from '@angular/core';

export interface UserProfile {
  personal: {
    firstName: string;
    lastName: string;
    dateOfBirth: string;
  };
  contact: {
    email: string;
    phone: string;
  };
  address: {
    street: string;
    city: string;
    state: string;
    zipCode: string;
  };
  preferences: {
    newsletter: boolean;
    notifications: boolean;
  };
}

@Injectable({
  providedIn: 'root'
})
export class ProfileService {
  // Simulate getting current user profile
  getCurrentProfile(): UserProfile {
    return {
      personal: {
        firstName: 'Emma',
        lastName: 'Wilson',
        dateOfBirth: '1995-03-15'
      },
      contact: {
        email: 'emma.wilson@example.com',
        phone: '555-1234'
```

```
      },
      address: {
        street: '456 Oak Avenue',
        city: 'Portland',
        state: 'OR',
        zipCode: '97201'
      },
      preferences: {
        newsletter: true,
        notifications: false
      }
    };
  }

  // Simulate saving profile
  saveProfile(profile: UserProfile): boolean {
    console.log('Saving profile to server:', profile);
    // Simulate API call
    return true;
  }
}
```

Component (profile-editor.component.ts):

```
import { Component, OnInit } from '@angular/core';
import { NgForm } from '@angular/forms';
import { ProfileService, UserProfile } from './profile.service';

@Component({
  selector: 'app-profile-editor',
  templateUrl: './profile-editor.component.html',
  styles: [`
    .form-section {
      margin: 20px 0;
      padding: 15px;
      border: 1px solid #ddd;
      border-radius: 5px;
    }
    .form-group {
      margin: 10px 0;
    }
    label {
      display: inline-block;
      width: 150px;
      font-weight: bold;
```

```
    }
    input, select {
      width: 300px;
      padding: 5px;
    }
    .button-group {
      margin-top: 20px;
    }
    button {
      margin-right: 10px;
      padding: 10px 20px;
    }
    .message {
      padding: 10px;
      margin: 10px 0;
      border-radius: 4px;
    }
    .success { background-color: #d4edda; color: #155724; }
    .error { background-color: #f8d7da; color: #721c24; }
    .section-invalid {
      border-left: 3px solid #dc3545;
    }
  `]
})
export class ProfileEditorComponent implements OnInit {
  profile: UserProfile = {
    personal: { firstName: '', lastName: '', dateOfBirth: '' },
    contact: { email: '', phone: '' },
    address: { street: '', city: '', state: '', zipCode: '' },
    preferences: { newsletter: false, notifications: false }
  };

  originalProfile: UserProfile | null = null;
  isSaving = false;
  saveSuccess = false;
  saveError = false;

  states = ['CA', 'NY', 'TX', 'FL', 'OR', 'WA'];

  constructor(private profileService: ProfileService) {}

  ngOnInit() {
    // Load current profile
    this.profile = this.profileService.getCurrentProfile();
    // Keep original for reset
    this.originalProfile = JSON.parse(JSON.stringify(this.profile));
  }
```

```
  onSave(form: NgForm) {
    if (form.valid) {
      this.isSaving = true;
      this.saveSuccess = false;
      this.saveError = false;

      // Simulate async save
      setTimeout(() => {
        const success = this.profileService.saveProfile(this.profile);
        this.isSaving = false;

        if (success) {
          this.saveSuccess = true;
          // Update original profile
          this.originalProfile = JSON.parse(JSON.stringify(this.profile));
        } else {
          this.saveError = true;
        }
      }, 1000);
    }
  }

  onReset(form: NgForm) {
    if (this.originalProfile) {
      this.profile = JSON.parse(JSON.stringify(this.originalProfile));
      form.reset(this.profile);
    }
  }

  onCancel() {
    console.log('Cancelled - navigating away');
  }
}
```

Template (profile-editor.component.html):

```
<div>
  <h2>Edit Profile</h2>

  <form #profileForm="ngForm">
    <!-- Personal Information -->
    <div class="form-section"
         [class.section-invalid]="personalGroup.invalid && personalGroup.touched">
      <h3>Personal Information</h3>
```

```
    <div ngModelGroup="personal" #personalGroup="ngModelGroup">
      <div class="form-group">
        <label>First Name:</label>
        <input type="text"
               name="firstName"
               [(ngModel)]="profile.personal.firstName"
               required
               minlength="2">
      </div>

      <div class="form-group">
        <label>Last Name:</label>
        <input type="text"
               name="lastName"
               [(ngModel)]="profile.personal.lastName"
               required
               minlength="2">
      </div>

      <div class="form-group">
        <label>Date of Birth:</label>
        <input type="date"
               name="dateOfBirth"
               [(ngModel)]="profile.personal.dateOfBirth"
               required>
      </div>
    </div>
  </div>

  <!-- Contact Information -->
  <div class="form-section"
       [class.section-invalid]="contactGroup.invalid && contactGroup.touched">
    <h3>Contact Information</h3>
    <div ngModelGroup="contact" #contactGroup="ngModelGroup">
      <div class="form-group">
        <label>Email:</label>
        <input type="email"
               name="email"
               [(ngModel)]="profile.contact.email"
               required
               email>
      </div>

      <div class="form-group">
        <label>Phone:</label>
        <input type="tel"
               name="phone"
```

```
               [(ngModel)]="profile.contact.phone"
               required
               pattern="[0-9]{3}-[0-9]{4}">
        <small> (Format: 555-1234)</small>
      </div>
    </div>
  </div>

  <!-- Address -->
  <div class="form-section"
       [class.section-invalid]="addressGroup.invalid && addressGroup.touched">
    <h3>Address</h3>
    <div ngModelGroup="address" #addressGroup="ngModelGroup">
      <div class="form-group">
        <label>Street:</label>
        <input type="text"
               name="street"
               [(ngModel)]="profile.address.street"
               required>
      </div>

      <div class="form-group">
        <label>City:</label>
        <input type="text"
               name="city"
               [(ngModel)]="profile.address.city"
               required>
      </div>

      <div class="form-group">
        <label>State:</label>
        <select name="state"
                [(ngModel)]="profile.address.state"
                required>
          <option value="">Select State</option>
          <option *ngFor="let state of states" [value]="state">
            {{ state }}
          </option>
        </select>
      </div>

      <div class="form-group">
        <label>Zip Code:</label>
        <input type="text"
               name="zipCode"
               [(ngModel)]="profile.address.zipCode"
               required
```

```
               pattern="[0-9]{5}">
      </div>
    </div>
  </div>

  <!-- Preferences -->
  <div class="form-section">
    <h3>Preferences</h3>
    <div ngModelGroup="preferences">
      <div class="form-group">
        <label>
          <input type="checkbox"
                 name="newsletter"
                 [(ngModel)]="profile.preferences.newsletter">
          Subscribe to newsletter
        </label>
      </div>

      <div class="form-group">
        <label>
          <input type="checkbox"
                 name="notifications"
                 [(ngModel)]="profile.preferences.notifications">
          Enable email notifications
        </label>
      </div>
    </div>
  </div>

  <!-- Messages -->
  <div *ngIf="isSaving" class="message">
    Saving profile...
  </div>

  <div *ngIf="saveSuccess" class="message success">
    Profile updated successfully!
  </div>

  <div *ngIf="saveError" class="message error">
    Failed to save profile. Please try again.
  </div>

  <!-- Buttons -->
  <div class="button-group">
    <button type="button"
            (click)="onSave(profileForm)"
            [disabled]="!profileForm.valid || isSaving">
```

```
        {{ isSaving ? 'Saving...' : 'Save Changes' }}
      </button>

      <button type="button"
              (click)="onReset(profileForm)"
              [disabled]="isSaving">
        Reset
      </button>

      <button type="button"
              (click)="onCancel()">
        Cancel
      </button>
    </div>

    <!-- Debug Info (remove in production) -->
    <div style="margin-top: 20px; padding: 10px; background: #f5f5f5;">
      <strong>Form Status:</strong>
      Valid: {{ profileForm.valid }} |
      Dirty: {{ profileForm.dirty }} |
      Touched: {{ profileForm.touched }}
    </div>
  </form>
</div>
```

Output:

When component loads: - Form displays Emma Wilson's profile with all fields pre-filled - All sections are valid (no red borders) User modifies: - First Name: "Emily" (changes from Emma) - Phone: "555-5678" (changes from 555-1234) Clicks "Save Changes": 1. Button shows "Saving..." 2. Message: "Saving profile..." 3. After 1 second: - Console: Saving profile to server: {personal: {firstName: "Emily", ...}, contact: {phone: "555-5678", ...}, ...} - Success message: "Profile updated successfully!" - Form status: Valid: true | Dirty: true | Touched: true User clicks "Reset": - Form reverts to original loaded values - First Name: back to "Emma" - Phone: back to "555-1234"

6. Practical Task

Task: Build a Product Registration Form

Requirements:

1. **Create a Product Registration System** with the following sections:
   - **Product Details** (ngModelGroup="product"):
     - Product Name (required, min 3 characters)
     - Category (dropdown: Electronics, Clothing, Books, Food)
     - Price (required, number, min 0)
     - SKU (required, pattern: 3 letters + 4 numbers, e.g., ABC1234)
   - **Inventory** (ngModelGroup="inventory"):
     - Quantity (required, number, min 0)
     - Warehouse Location (required)
     - Reorder Level (number, min 0)
   - **Status** (ngModelGroup="status"):
     - Active (checkbox)
     - Featured (checkbox)
2. **Prepopulate Form** with a product in "edit mode":

```
{
  product: {
    name: 'Wireless Mouse',
    category: 'Electronics',
    price: 29.99,
    sku: 'ELC1001'
  },
  inventory: {
    quantity: 150,
    location: 'Warehouse A',
    reorderLevel: 20
  },
  status: {
    active: true,
```

```
      featured: false
    }
  }
```

3. **Implement Features**:
    - Display validation errors for each group
    - Show different styling for invalid sections
    - Add "Save Product" button (disabled if form invalid)
    - Add "Reset to Original" button
    - Add "Clear Form" button (resets to empty)
    - Show success/error messages after submission
    - Log the complete form structure to console on save
4. **Additional Challenge**:
    - Add calculated field showing "Total Value" (Price × Quantity)
    - Update automatically when price or quantity changes
    - Display warning if quantity is below reorder level

Expected Console Output on Save:

```
{
  product: {
    name: "Gaming Keyboard",
    category: "Electronics",
    price: 89.99,
    sku: "ELC2045"
  },
  inventory: {
    quantity: 75,
    location: "Warehouse B",
    reorderLevel: 15
  },
  status: {
    active: true,
    featured: true
```

```
  }
}
Total Inventory Value: $6749.25
```

Validation Rules Summary:

- Product Name: required, minlength=3
- Category: required
- Price: required, min=0
- SKU: required, pattern="[A-Z]{3}[0-9]{4}"
- Quantity: required, min=0
- Warehouse Location: required
- Reorder Level: min=0

---

7. Summary

Key Concepts Covered

Concept	Description	Use Case
Prepopulation	Loading existing data into form fields	Edit forms, default values, profile management
One-Way Binding	`[ngModel]` or `(ngModelChange)` separately	Controlled updates, read-only fields, custom logic
ngModelGroup	Group related form controls	Complex forms, nested data, section validation
Form Submission	`(ngSubmit)` event handling	Processing form data, API calls, validation
Form Reset	`form.reset()` method	Clear form, restore defaults, after submission

Best Practices

1. **Prepopulation:**
   - Load data in `ngOnInit()` lifecycle hook
   - Use services to fetch data
   - Keep a copy of original data for reset functionality
2. **Data Binding:**
   - Use two-way binding `[(ngModel)]` for simple cases
   - Split into one-way bindings when custom logic is needed
   - Consider performance impact of binding
3. **Form Organization:**
   - Group related fields with `ngModelGroup`
   - Use meaningful group names that reflect data structure
   - Validate groups independently
4. **Submission Handling:**
   - Always check `form.valid` before processing
   - Provide visual feedback during submission
   - Handle success and error states
   - Reset form after successful submission if appropriate
5. **User Experience:**
   - Disable submit button when form is invalid
   - Show clear success/error messages
   - Provide reset/cancel options
   - Indicate loading states during async operations

Common Patterns

Edit Mode Pattern:

```
ngOnInit() {
  this.loadData();
  this.keepOriginalCopy();
}
```

Controlled Update Pattern:

```
<input [ngModel]="value"
       (ngModelChange)="onValueChange($event)">
```

Grouped Validation Pattern:

```
<div ngModelGroup="section" #sectionGroup="ngModelGroup">
  <!-- fields -->
  <div *ngIf="sectionGroup.invalid">Errors</div>
</div>
```

Async Submission Pattern:

```
onSubmit() {
  this.isSubmitting = true;
  // async operation
  this.isSubmitting = false;
}
```

What's Next?

In the next module, we'll explore **Reactive Forms**, which provide:

- More control over form state
- Dynamic form creation
- Complex validation scenarios
- Better testability
- Programmatic form manipulation

> **Module Complete!** You now understand advanced template-driven form features including prepopulation, one-way binding, form grouping, and proper submission handling.